

06 - File IO

哈尔滨工业大学 黄子扬

2026 年 6 月 6 日

Contents

1	File Systems Paths / 文件系统与路径	2
2	The open() Function / 打开文件	2
2.1	Opening Modes / 打开模式	2
2.2	Writing and Closing / 写入与关闭	2
3	Reading Files / 读取文件	2
4	Working with CSV Files / CSV 文件处理	3
5	Error Handling Debugging / 异常处理与调试	3
5.1	Error Types / 错误类型	3
5.2	The Try-Except Block / 捕获异常	4
6	PEP-8 Style Guide / 编程规范标准	4

1 File Systems Paths / 文件系统与路径

【Concept / 核心概念】1

Hierarchy and Paths / 层级与路径 Files are organized in a hierarchy of **directories** (folders). A **file path** is a string representing the location of a file starting from the root.

文件存储在目录（文件夹）层级中。** 文件路径 ** 是从根目录开始定位文件的字符串 [1]。

【Warning / 避坑指南】1

OS Differences / 操作系统差异 Path slashes depend on your OS:

- **Windows:** Uses backslashes (\), e.g., C:\Users\hello.txt.
- **macOS / Linux:** Uses forward slashes (/), e.g., /Users/hello.txt [2].

2 The open() Function / 打开文件

2.1 Opening Modes / 打开模式

【Concept / 核心概念】1

The mode Parameter / 模式参数 Before reading or writing, you must use `open()` to create a file object [2].

- **'r'** (Read): Read only (default). / 只读。
- **'w'** (Write): Overwrites existing file or creates a new one. / 覆写（会清空原内容）[3]。
- **'a'** (Append): Adds content to the end of the file. / 追加（在末尾添加内容）[3]。

2.2 Writing and Closing / 写入与关闭

【Example / 实战案例】1

Writing to a File / 写入实操

```
my_file = open(file='sample.txt', mode='w')
my_file.write('Hello file!')
my_file.write('\n') # Manually add a newline / 必须手动加换行符 [4, 5]
my_file.write('This is me!')
my_file.close()      # Always close to flush data / 必须关闭以保存数据 [3]
```

3 Reading Files / 读取文件

- **.read():** Reads the **entire** file as one string. / 将整个文件读入一个字符串 [4]。
- **.readline():** Reads just **one line**. / 仅读取一行 [6]。

- `.readlines()`: Reads the whole file and returns a **list of strings**. / 读取整个文件并返回字符串列表 [6]。

【Example / 实战案例】1

Iterating over Lines / 逐行遍历案例

```
my_file = open(file='sample.txt', mode='r')
for line in my_file:    # Most memory-efficient way / 最省内存的遍历方式 [6]
    print(line)
my_file.close()
```

4 Working with CSV Files / CSV 文件处理

【Concept / 核心概念】1

What is CSV? / 什么是 CSV? CSV stands for **Comma Separated Values**. It is a simple but powerful way to store data in a table format [4].

【Example / 实战案例】1

Using DictReader / 使用字典读取器 `csv.DictReader` is very useful because it treats each row as a dictionary where keys are column names.

`DictReader` 将每一行映射为一个字典，非常方便处理 [7]。

```
import csv
csv_file = open("sample.csv", "r")
reader = csv.DictReader(csv_file)
for row in reader:
    print(row) # Output: {'id': '1', 'name': 'Rose', ...}
csv_file.close()
```

5 Error Handling Debugging / 异常处理与调试

5.1 Error Types / 错误类型

- **SyntaxError**: Caught by the IDE **before** execution (e.g., forgetting a colon :). / 语法错误，代码无法运行 [8]。
- **Exceptions**: Occur **during** execution (e.g., `IndexError` for out-of-range lists). / 运行时异常，需妥善处理 [8]。

5.2 The Try-Except Block / 捕获异常

【Example / 实战案例】1

Handling User Input / 处理用户输入案例

```
user_input = input("Tell me your name: ")
try:
    user_input = float(user_input) # Try to convert to number
    print("That's a number, not a name!")
except:
    # Runs if 'try' block fails
    print("Thank you,", user_input) [9]
```

6 PEP-8 Style Guide / 编程规范标准

【Concept / 核心概念】1

Professional Coding / 像专家一样写代码 Writing easy-to-understand code is crucial. Follow **PEP-8** conventions [10].

- **Whitespaces:**
 - **DO** put spaces after commas and around operators: `2 + 2` [11].
 - **DO NOT** put spaces between a function name and parentheses: `func(x)` [11].
- **Blank Lines:**
 - Separate function definitions by **two** blank lines [12].
 - Separate logic blocks (loops, if-statements) by **one** blank line [12].
 - **Rule of thumb:** No more than 4 lines without a blank line [12].
- **Comments:** DO NOT add self-explanatory comments (e.g., `x = x + 1 # increment x is bad`) [13, 14].